Module 6: Consensus and Validation

Multi-Layer Security & Validation Framework

Validation Architecture Overview

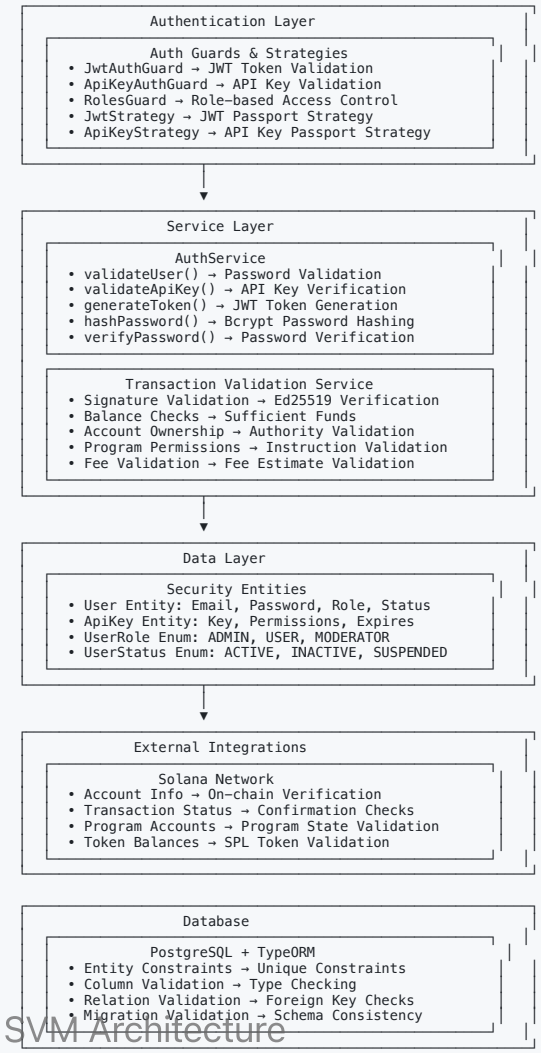
Validation Layers

- **Authentication**: JWT tokens, API keys, role-based access
- **Input Validation**: DTO validation, data transformation
- **Business Logic**: Transaction validation, program execution
- **External Checks**: Solana network validation
- **Database**: TypeORM constraints and relationships

Security Principles

- **Defense in Depth**: Multiple validation layers
- **Fail-Fast**: Early validation prevents unnecessary processing
- **Comprehensive Coverage**: All data paths validated
- **Error Transparency**: Clear error messages and codes

System Architecture



Validation Pipeline

Request Processing Flow

```
1. 🔐 Auth Guards → JWT/API Key Check
2. ✅ Input Validation → DTO/Class Validation
3. 🎯 Business Rules → Service Layer Validation
4. 🌐 External Checks → Solana Network Validation
5. 💾 Database Ops → TypeORM Constraints
6. 📤 Response → Validated Data
```

Pipeline Benefits

- **Sequential Processing**: Each layer builds on previous validation
- **Early Failure**: Invalid requests rejected quickly
- **Resource Efficiency**: Only valid requests reach expensive operations
- **Audit Trail**: Complete validation history for debugging

Authentication & Authorization

Auth Guards Implementation

```
@Injectable()
export class JwtAuthGuard extends AuthGuard('jwt') {
  canActivate(context: ExecutionContext): boolean {
    const request = context.switchToHttp().getRequest();
    const token = this.extractTokenFromHeader(request);

    if (!token) {
      throw new UnauthorizedException('No token provided');
    }

    try {
      const payload = this.jwtService.verify(token);
      request.user = payload;
      return true;
    } catch (error) {
      throw new UnauthorizedException('Invalid token');
    }
  }
```

Transaction Validation

Signature Validation

```
validateTransactionSignature(transaction: Transaction): boolean {
  try {
    // Verify all required signatures
    for (const signature of transaction.signatures) {
      const message = transaction.serializeMessage();
      const isValid = nacl.sign.detached.verify(
        message,
        signature.signature,
        signature.publicKey.toBytes()
      );

      if (!isValid) {
        throw new BadRequestException('Invalid transaction signature');
      }
    }
    return true;
  } catch (error) {
    throw new BadRequestException('Transaction signature validation failed');
  }
```

Program Execution Validation

SVM Runtime Validation

```
validateProgramExecution(programId: PublicKey, instruction: TransactionInstruction): void {
  // Validate program exists and is executable
  const accountInfo = await this.connection.getAccountInfo(programId);
  if (!accountInfo) {
    throw new NotFoundException('Program account not found');
  }

  if (!accountInfo.executable) {
    throw new BadRequestException('Account is not executable');
  }

  // Check compute unit limits
  const estimatedCU = this.estimateComputeUnits(instruction);
  if (estimatedCU > MAX_COMPUTE_UNITS) {
    throw new BadRequestException('Instruction exceeds compute unit limit');
  }

  // Validate instruction data format
  this.validateInstructionData(instruction);
}
```

Smart Account Validation

Rule-Based Validation

```
async validateSmartAccountTransaction(
  smartAccount: SmartAccount,
  transaction: Transaction
): Promise<ValidationResult> {

  // Check account status
  if (smartAccount.status !== SmartAccountStatus.ACTIVE) {
    return { valid: false, reason: 'Smart account is not active' };
  }

  // Validate daily spending limit
  const dailySpent = await this.getDailySpending(smartAccount.address);
  const transactionAmount = this.calculateTransactionAmount(transaction);

  if (dailySpent + transactionAmount > smartAccount.rules.maxDailySpend) {
    return { valid: false, reason: 'Daily spending limit exceeded' };
  }

  // Check program whitelist
  const allowedPrograms = new Set(smartAccount.rules.allowedPrograms);
  for (const ix of transaction.instructions) {
    if (!allowedPrograms.has(ix.programId.toString())) {
      return { valid: false, reason: 'Program not in whitelist' };
    }
  }
```

Input Validation & DTOs

Class Validator Integration

```
import { IsNotEmpty, IsEmail, IsString, MinLength } from 'class-validator';

export class CreateUserDto {
  @IsNotEmpty()
  @IsEmail()
  email: string;

  @IsNotEmpty()
  @IsString()
  @MinLength(8)
  password: string;

  @IsNotEmpty()
  @IsString()
  role: UserRole;
}

export class CreateTransactionDto {
  @IsNotEmpty()
  @IsString()
  fromAddress: string;

  @IsNotEmpty()
  @IsString()
  toAddress: string;

  @IsNotEmpty()
  @Min(0)
  amount: number;

  @IsOptional()
```

Error Handling & Responses

Standardized Error Responses

```
export class ValidationErrorResponse {
  statusCode: number;
  message: string | string[];
  error: string;
  timestamp: string;
  path: string;
}

// Example error responses
const errors = {
  400: { statusCode: 400, message: 'Validation failed', error: 'Bad Request' },
  401: { statusCode: 401, message: 'Unauthorized', error: 'Unauthorized' },
  403: { statusCode: 403, message: 'Forbidden', error: 'Forbidden' },
  404: { statusCode: 404, message: 'Not found', error: 'Not Found' },
  409: { statusCode: 409, message: 'Conflict', error: 'Conflict' }
};
```


Exception Filters

Database Validation

TypeORM Entity Validation

```
@Entity()
export class User {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column({ unique: true })
  @Index()
  email: string;

  @Column()
  passwordHash: string;

  @Column({
    type: 'enum',
    enum: UserRole,
    default: UserRole.USER
  })
  role: UserRole;

  @Column({
    type: 'enum',
    enum: UserStatus,
    default: UserStatus.ACTIVE
  })
  status: UserStatus;

  @CreateDateColumn()
  createdAt: Date;

  @UpdateDateColumn()
```

Key Takeaways

Validation Framework Benefits

- **Multi-Layer Security**: Authentication, authorization, and business rules
- **Comprehensive Coverage**: All data inputs and operations validated
- **Performance Optimized**: Early validation prevents unnecessary processing
- **Developer Experience**: Clear error messages and validation feedback

SVM Consensus Advantages

- **Cryptographic Security**: Ed25519 signature validation
- **Parallel Validation**: High-throughput transaction processing
- **State Consistency**: Atomic validation across all operations
- **Network Security**: On-chain verification of all state changes